

Universidade de Brasília
Departamento de Ciência da Computação
Organização e Arquitetura de Computadores – Grupo 01, 2026/1
Prof. Marcus Vinicius Lamar

Laboratório 01 - Assembly - RISC-V

Guilherme Nascimento - 222001449
Letícia Brito - 242039381
Halycia de Oliveira Fonseca - 221037625
Eduardo Pereira de Sousa - 231018937
Matheus Chagas Lopes - 222011599
Pedro Fernandes de Oliveira - 231006177

Brasília, 2026

Parte 1) Questão 1.1 - Execução do Algoritmo `sort.s`

A primeira etapa do laboratório consistiu na execução direta do algoritmo de ordenação *Insertion Sort* fornecido, utilizando o vetor padrão de 32 elementos. A medição das instruções foi realizada através da ferramenta *Instruction Statistics* do simulador RARS.

a) Ordenação Crescente:

- **Total de Instruções Executadas:** 5.806 instruções.
- **Tamanho do Código Executável (`.text`):** 276 bytes.
- **Tamanho da Memória de Dados (`.data`):** 128 bytes.

b) **Ordenação Decrescente:** Para inverter a lógica de ordenação, a instrução de desvio condicional do laço interno foi modificada de `bge t4, t3, exit2` para `ble t4, t3, exit2`. Os resultados obtidos após a execução foram:

- **Total de Instruções Executadas:** 4677 instruções.
- **Tamanho do Código Executável (`.text`):** 276 bytes.
- **Tamanho da Memória de Dados (`.data`):** 128 bytes.

Parte 1) Questão 1.2 a - Derivação das Equações de Tempo de Execução

Para obter as equações de tempo, contamos o número exato de instruções executadas em cada trecho do código, para os dois casos extremos de entrada. Como o processador opera a 50 MHz com $CPI = 1$, cada instrução consome exatamente $T = 20$ ns.

Caso 1 - Vetor já ordenado: $V_o[n] = \{1, 2, 3, \dots, n\}$

Este é o melhor caso do *Insertion Sort*. Como o vetor já está ordenado, o laço interno `for2` encontra imediatamente que não há necessidade de troca e encerra sua execução logo na primeira comparação de cada iteração do laço externo. Nenhum SWAP é realizado.

Contagem de instruções por trecho

- **Início:** executado uma única vez ao entrar no procedimento, realiza o salvamento dos registradores na pilha e a inicialização das variáveis de controle. $\rightarrow 9$ instruções fixas.
- **Laço `for1`:** itera n vezes, controlando o índice externo. A cada iteração executa o teste de saída, o decremento do índice interno, o incremento do índice externo e o salto de retorno. Mais 1 instrução para o teste final que encerra o laço. $\rightarrow 4n + 1$ instruções.

- **Laço for2:** no melhor caso, executa apenas os carregamentos e a comparação que constata que o vetor já está ordenado, saindo imediatamente. Isso ocorre uma vez por iteração do **for1**. $\rightarrow 6n$ instruções.
- **Fim:** executado uma única vez ao sair do procedimento, restaura os registradores salvos e retorna. $\rightarrow 7$ instruções fixas.

Equação do tempo de execução

$$t_o(n) = (10n + 17) \times 20 \text{ ns} = 200n + 340 \text{ ns}$$

Caso 2 - Vetor inversamente ordenado: $V_i[n] = \{n, n - 1, \dots, 2, 1\}$

Este é o pior caso do *Insertion Sort*. Como cada elemento está na posição mais distante possível da sua posição final, o laço interno **for2** percorre todo o caminho de volta e realiza um SWAP a cada passo. Na iteração i do laço externo, o laço interno executa exatamente i vezes, totalizando $\frac{n(n-1)}{2}$ iterações ao longo de todo o procedimento.

Contagem de instruções por trecho

- **Início:** idêntico ao melhor caso. $\rightarrow 9$ instruções fixas.
- **Laço for1:** idêntico ao melhor caso. $\rightarrow 4n + 1$ instruções.
- **Laço for2 com SWAP:** no pior caso, cada iteração do **for2** executa o teste, os carregamentos, a comparação (não satisfeita), a chamada ao SWAP com todas as suas instruções internas, o decremento do índice e o salto de retorno ao início do laço, totalizando 18 instruções por iteração.

O número total de iterações do **for2** ao longo de todo o SORT é:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

Portanto, o total de instruções do **for2** é:

$$18 \times \frac{n(n-1)}{2} = 9n(n-1) = 9n^2 - 9n$$

$\rightarrow 9n^2 - 9n$ instruções.

- **Fim:** idêntico ao melhor caso. $\rightarrow 7$ instruções fixas.

Equação do tempo de execução

$$t_i(n) = (9n^2 - 5n + 17) \times 20 \text{ ns} = 180n^2 - 100n + 340 \text{ ns}$$

Questão 1.2 b

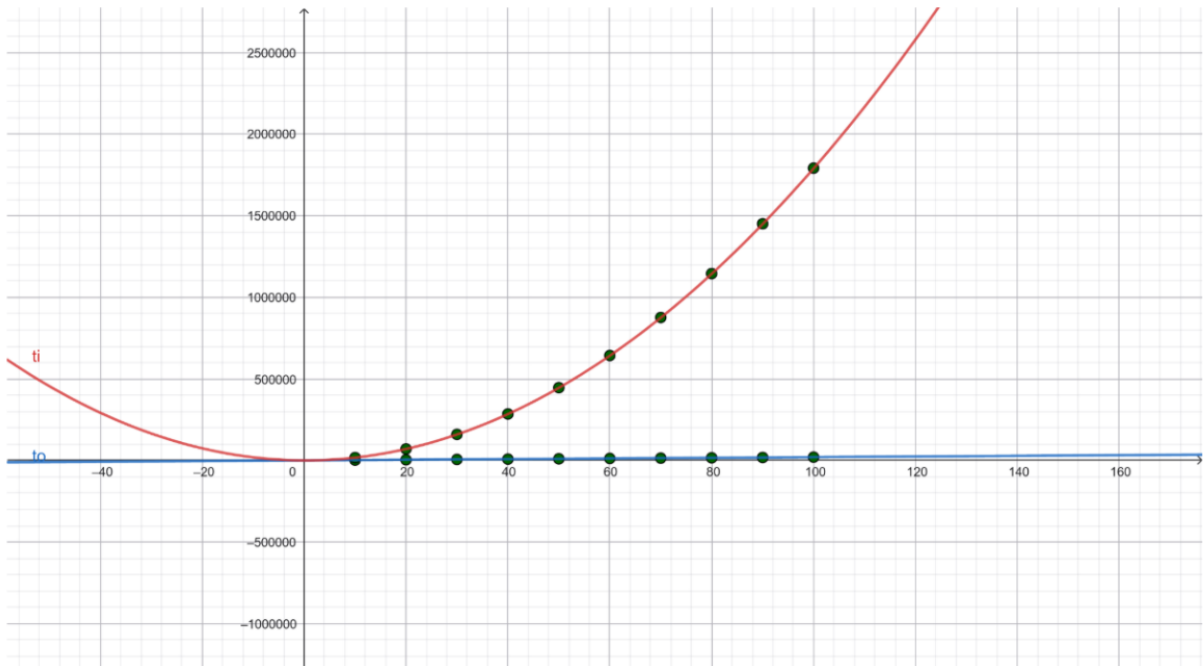


Figura 1: Diferença entre os casos de entrada

O gráfico mostra a diferença entre os dois casos de entrada do algoritmo de ordenação em um processador RISC-V de 50 MHz com $CPI = 1$.

- **Comportamento de $t_o(n)$:** O tempo de execução para o vetor já ordenado cresce de forma linear com o número de elementos. Isso ocorre porque, neste caso, o laço interno `for2` encerra imediatamente a cada iteração do laço externo, sem realizar nenhuma troca. O algoritmo percorre o vetor apenas uma vez por elemento, executando um número mínimo de instruções proporcional a n . No gráfico, essa curva aparece praticamente colada ao eixo x . Para $n = 100$, o tempo de execução é de apenas 20.280 ns.
- **Comportamento de $t_i(n)$:** O tempo de execução para o vetor inversamente ordenado cresce de forma quadrática com n . Este é o pior caso do algoritmo: cada elemento inserido precisa ser comparado e trocado com todos os elementos anteriores, pois nenhum está na posição correta. O número total de operações cresce proporcionalmente a $\frac{n(n-1)}{2}$, o que gera a parábola observada no gráfico. Para $n = 100$, o tempo chega a 1.792.380 ns.

Parte 2 - Compilação Cruzada e Análise de Otimizações (GCC)

Esta etapa do laboratório tem como objetivo explorar o processo de compilação cruzada (*cross-compiling*) e avaliar o impacto prático das heurísticas de otimização de com-

piladores na geração de código Assembly. Diferente da escrita manual de instruções abordada nas seções anteriores, o foco aqui é compreender como programas escritos em linguagem de alto nível (C) são traduzidos para a arquitetura alvo RISC-V (RV32IMF).

Utilizando a ferramenta GCC através do ambiente *Compiler Explorer*, os códigos-fonte foram convertidos em linguagem de montagem. Ao longo desta seção, são detalhadas as adaptações necessárias para tornar o código auto-gerado compatível com o simulador RARS. Além disso, é conduzida uma análise comparativa rigorosa sobre como diferentes diretivas de otimização (-O0, -O1, -O3 e -Os) afetam diretamente o desempenho do processador, avaliando métricas cruciais como a contagem total de instruções, a redução de ciclos de *clock* e o tamanho (em *bytes*) do código de máquina gerado.

2.2) Para fazer com que o código funcionasse no System-Rars, foram necessárias algumas mudanças:

- Identificar o `.data` e `.text`
- Remover o chamado da função inexistente `printf`
- Remover o chamado da função inexistente `putchar`
- Colocar a `main` após o `.text`
- Substituir o `call` da `printf` e da `putchar` por `ecall`
- Remover a instrução `jr ra` da `main`
- Corrigir a instrução `lla` por `la`

Obs: Como os códigos para essa parte do laboratório estão disponíveis no arquivo disponibilizado pelo professor Lamar, acreditamos que seria redundante colocá-lo no relatório novamente. Por isso, decidimos colocar apenas o resultado.

2.3)

Diretiva	Total de Instruções	Tamanho em bytes
-O0	2174	1132
-O3	4399	876
-Os	4399	920

2.4)

Função	-O0 (ciclos)	-O1 (ciclos)	Redução
f1	14	4	71,40%
f2	14	4	71,40%
f3	14	4	71,40%
f4	14	4	71,40%
f5	14	4	71,40%

Nota-se que independentemente da operação escrita em C (soma, multiplicação ou deslocamento), o compilador identificou que todas resultavam no mesmo valor final ($x \times 4$). No Assembly gerado, todas foram traduzidas para a instrução `slli` (*Shift Left Logical Immediate*), que é a forma mais eficiente de realizar essa operação em arquitetura RISC-V.

O que muda em `-O0` e `-O1` é a questão da otimização. Em `-O0` não há otimização, há um código literal gerado pelo compilador. Já no caso do `-O1` há uma simplificação das funções de maneira que não necessite acessar memória para salvar variáveis locais.

Por fim, cada otimização tem o propósito de melhorar a performance do programa em relação a tempo e uso de memória. A `-Os` foca em economia de bytes enquanto o `-O3` foca em rodar mais rápido.

Parte 3 - Desenho de Gráficos, Cálculo de Raízes e Análise de Desempenho

Esta seção detalha a implementação adotada para cumprir os requisitos da Parte 3 do laboratório, desenvolvida em linguagem Assembly RISC-V, utilizando a extensão de ponto flutuante de precisão simples (RV32F) e a infraestrutura de chamadas de sistema fornecida pelo ambiente customizado da UnB (`SYSTEMv24.s` e `MACROsv24.s`).

Destaque de Implementação: O Núcleo Matemático (baskara)

Abaixo, destaca-se o trecho mais crítico do código desenvolvido: o cálculo do discriminante (Δ) e a avaliação condicional utilizando a extensão de ponto flutuante do RISC-V (RV32F). Este fragmento demonstra a conversão de tipos (`fcvt.s.w`), operações aritméticas de precisão simples e o controle de fluxo baseado em flags da unidade de ponto flutuante (`flt.s`).

baskara:

```
# Calcula Delta = b^2 - 4ac
fmul.s ft0, fa1, fa1      # ft0 = b^2
li t0, 4
fcvt.s.w ft1, t0
fmul.s ft1, ft1, fa0      # ft1 = 4 * a
fmul.s ft1, ft1, fa2      # ft1 = 4ac
fsub.s ft0, ft0, ft1      # ft0 = b^2 - 4ac (Delta)

# Prepara constante 0 e avalia se Delta < 0
fmv.w.x ft2, zero        # ft2 = 0.0
flt.s t1, ft0, ft2       # t1 = 1 se Delta < 0.0

# Prepara divisor 2a e negativo de b (-b)
li t0, 2
fcvt.s.w ft2, t0          # Converte int 2 para float
fmul.s ft2, ft2, fa0      # ft2 = 2a
```

```
fneg.s ft3, fa1          # ft3 = -b
```

```
bnez t1, raizes_complexas # Salta se Delta for negativo
```

A importância deste trecho reside na forma como o hardware gerencia a transição entre registradores lógicos (`t0`, `t1`) e registradores de ponto flutuante (`ft0`, `fa1`). A instrução `flt.s` atua como uma ponte, avaliando uma condição no coprocessador matemático e sinalizando o resultado para a CPU principal tomar a decisão de salto (`bnez`), otimizando o uso do *pipeline*.

3.1 - Avaliação Polinomial: Procedimento `fx`

O procedimento `fx` foi projetado para avaliar matematicamente uma função polinomial do segundo grau:

$$f(x) = ax^2 + bx + c$$

Em conformidade com as convenções de chamada, o procedimento recebe o argumento x no registrador `fa0`, e os coeficientes a , b e c nos registradores `fa1`, `fa2` e `fa3`, respectivamente. O resultado final é retornado em `fa0`. A estratégia de implementação utilizou mapeamento direto:

- `fmul.s fs4, fs0, fs0`: computa o termo x^2 .
- `fmul.s fs1, fs1, fs0`: computa $a \times x^2$.
- `fmul.s fa0, fs2, fs3`: computa $b \times x$.
- Acúmulo com `fadd.s` consolida a expressão.

3.2 - Visualização Gráfica e Algoritmo de Auto-Escalamento

A renderização geométrica da parábola no *Bitmap Display* exigiu o desenho via `ecall 147 (Draw Line)`. Para assegurar que qualquer parábola fosse perfeitamente visível na tela de 320×240 pixels, foi desenvolvido um algoritmo de **auto-escalamento**:

- **Cálculo da Escala X:** O programa calcula a coordenada do vértice $X_v = -b/(2a)$. O fator de escala horizontal mapeia a janela matemática em torno desse eixo nos 160 pixels disponíveis para cada semi-eixo da tela.
- **Varredura e Escala Y:** O algoritmo realiza um *scan* prévio composto por 320 iterações. O programa armazena o maior valor absoluto de imagem $|f(x)|$ e define a escala vertical pela razão entre a metade da altura da tela (120 pixels) e esse valor, prevenindo *clipping*.
- **Traçado Contínuo:** O código armazena a coordenada do pixel imediatamente anterior e utiliza a chamada de sistema para ligar os pontos consecutivos, gerando uma curva perfeitamente contínua.

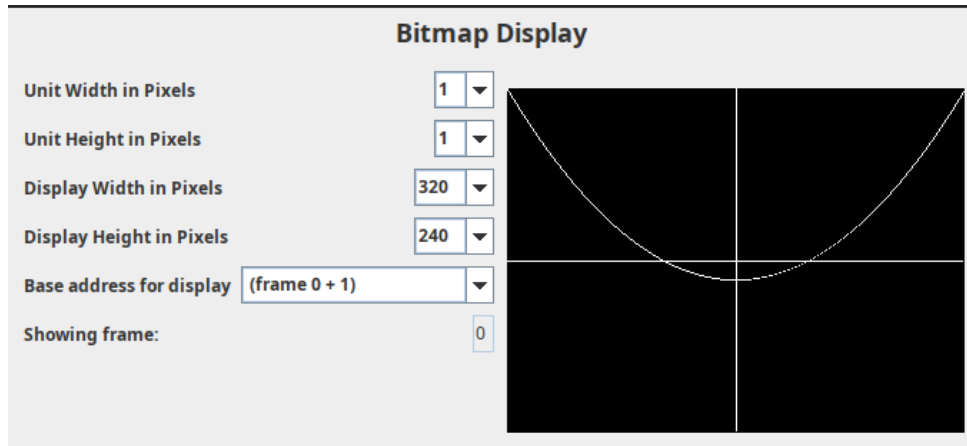


Figura 2: Renderização geométrica da função polinomial e eixos cartesianos

3.3 - Cálculo das Raízes: Procedimento baskara

O procedimento `baskara` implementa a solução analítica recebendo os coeficientes nos registradores de ponto flutuante, operando no seguinte fluxo:

- **Análise do Discriminante (Δ):** Calcula-se $\Delta = b^2 - 4ac$. O sinal de Δ é avaliado em comparação com zero por meio da instrução `flt.s`.
- **Raízes Reais ($\Delta \geq 0$):** Executa-se `fsqrt.s`. As raízes reais são calculadas, empurradas para a pilha (`sp`) e o registrador `a0` recebe o código 1 (Tipo Real).
- **Raízes Complexas ($\Delta < 0$):** O algoritmo isola a parte real e a parte imaginária. O par conjugado é salvo na pilha e o registrador `a0` recebe o código 2 (Tipo Complexo).

3.4 - Leitura Interativa de Dados e Interface Textual (`show`)

Para atender à especificação de execução contínua, o programa principal (`main`) foi reestruturado com um laço infinito. Utilizando as chamadas de sistema `ecall 4` (impressão de string) e `ecall 6` (leitura de float), o programa solicita interativamente os coeficientes a , b e c pelo console de entrada e saída (*Run I/O*). Após a leitura, as sub-rotinas são invocadas e o laço retorna ao início (`j main`), permitindo testes sucessivos sem reinicialização.

O procedimento `show` atua como a camada de apresentação. Ele extrai os dados da pilha e os formata no canto superior esquerdo do display seguindo estritamente as especificações visuais do roteiro:

- **Raízes Reais:** Formatadas como $R(1) = \text{Valor}$ e $R(2) = \text{Valor}$.
- **Raízes Complexas:** Formatadas em notação imaginária, exibindo $R(1) = \text{Parte Real} + \text{Parte Imag } i$ e $R(2) = \text{Parte Real} - \text{Parte Imag } i$.

A exibição gráfica dessa formatação é construída através das seguintes *syscalls*:

- `ecall 104`: Desenha os rótulos de texto estáticos (*strings* de formatação).
- `ecall 102`: Realiza o desenho dos valores em ponto flutuante precisão simples na tela.

Exemplo: Raízes Reais (Cenário A)

R(1) = +3.14159250E+00

R(2) = -3.14159250E+00

Instrucoes (I): 29

Ciclos (C): 27

CPI: +9.99985694E-01

Tempo (ms): 0

Exemplo: Raízes Complexas (Cenário C)

R(1) = -4.94999980E+00 +

2.95804004E+00 i

R(2) = -4.94999980E+00 -

2.95804004E+00 i

Instrucoes (I): 27

Ciclos (C): 29

CPI: +9.99987983E-01

Tempo (ms): 0

3.5 - Análise Analítica de Desempenho da Rotina `baskara`

O item 3.5 requisita o cálculo teórico do tempo de execução estrito da rotina `baskara`, assumindo um processador RISC-V de 1 GHz ($T_{clock} = 1$ ns), onde instruções inteiras consomem 1 ciclo e instruções de ponto flutuante (FP) consomem 10 ciclos.

Analisando o código Assembly desenvolvido, a rotina `baskara` possui um bloco base comum e uma bifurcação condicional dependente do valor do discriminante (Δ). Todas as instruções do tipo `f*` (`fmul`, `fadd`, `fsqrt`, etc.) foram contabilizadas como FP (10 ciclos).

1. Bloco Base (Comum a todos os fluxos): Calcula o Δ e prepara a bifurcação.

- Instruções Inteiras: 3 (`li`, `li`, `bnez`) $\rightarrow 3 \times 1 = 3$ ciclos.
- Instruções FP: 10 (`fmul.s`, `fcvt.s.w`, `fmul.s`, `fmul.s`, `fsub.s`, `fmv.w.x`, `flt.s`, `fcvt.s.w`, `fmul.s`, `fneg.s`) $\rightarrow 10 \times 10 = 100$ ciclos.
- *Subtotal Base = 103 ciclos.*

2. Caminho A - Raízes Reais ($\Delta \geq 0$):

- Instruções Inteiras: 3 (`addi`, `li`, `ret`) $\rightarrow 3 \times 1 = 3$ ciclos.
- Instruções FP: 7 (`fsqrt.s`, `fadd.s`, `fdiv.s`, `fsub.s`, `fdiv.s`, `fsw`, `fsw`) $\rightarrow 7 \times 10 = 70$ ciclos.
- **Total Raízes Reais:** $103 + 73 = 176$ ciclos $\times 1$ ns = **176** ns.

3. Caminho B - Raízes Complexas ($\Delta < 0$):

- Instruções Inteiras: 3 (`addi`, `li`, `ret`) $\rightarrow 3 \times 1 = 3$ ciclos.
- Instruções FP: 6 (`fdiv.s`, `fneg.s`, `fsqrt.s`, `fdiv.s`, `fsw`, `fsw`) $\rightarrow 6 \times 10 = 60$ ciclos.
- **Total Raízes Complexas:** $103 + 63 = 166$ ciclos $\times 1$ ns = **166 ns**.

Aplicação nos Cenários de Teste

Abaixo são apresentados os resultados teóricos de tempo de execução e a constatação visual gráfica para cada uma das equações testadas interativamente no RARS:

- a) **[1, 0, -9.86960440]:** $\Delta \approx 39.47$ (Raízes Reais). **Tempo de execução de baskara: 176 ns.**
- b) **[1, 0, 0]:** $\Delta = 0$ (Raízes Reais). **Tempo de execução de baskara: 176 ns.**
- c) **[1, 99, 2459]:** $\Delta = -35$ (Raízes Complexas conjugadas). **Tempo de execução de baskara: 166 ns.**
- d) **[1, -2468, 33762440]:** $\Delta = -128.960.576$ (Raízes Complexas). Apesar dos valores elevados, a performance da unidade lógica e aritmética de ponto flutuante do RISC-V é constante. **Tempo de execução de baskara: 166 ns.**
- e) **[0, 10, 100]:** Como $a = 0$, trata-se de uma reta ($y = 10x + 100$), não de uma parábola. O algoritmo de Bhaskara executa $\Delta = 100$ e entra no bloco de Raízes Reais. Contudo, ao realizar a divisão por $2a$, o processador realiza uma divisão por zero. O simulador atribui valor de *Infinity* (Infinito) ou *NaN* para as raízes geradas. **Tempo de execução de baskara: 176 ns.**

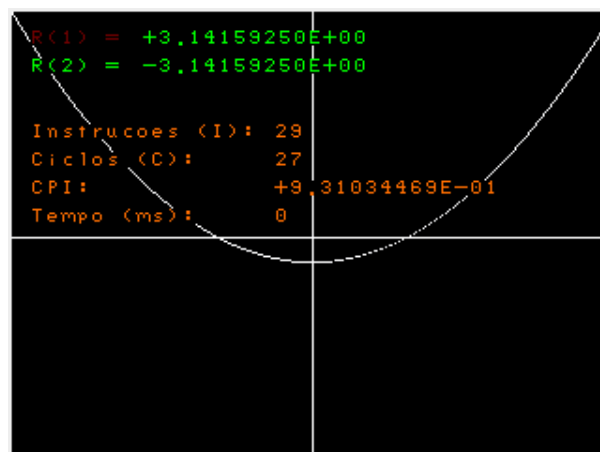


Figura 3: a) [1.0,0,-9.86960440]

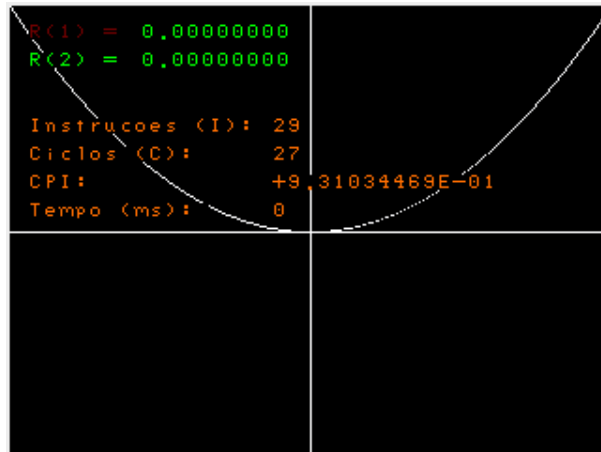


Figura 4: b) $[1, 0, 0]$

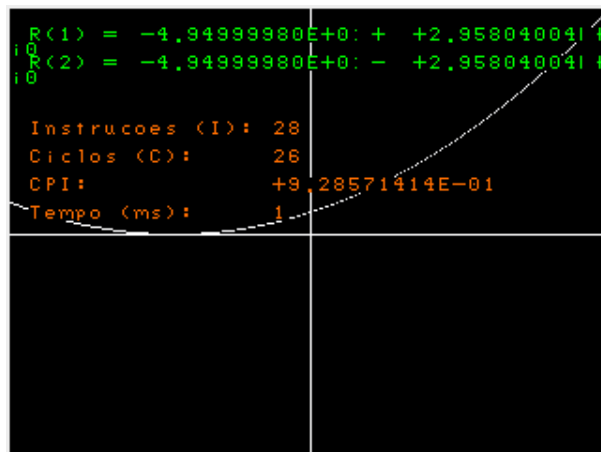


Figura 5: c) $[1, 99, 2459]$

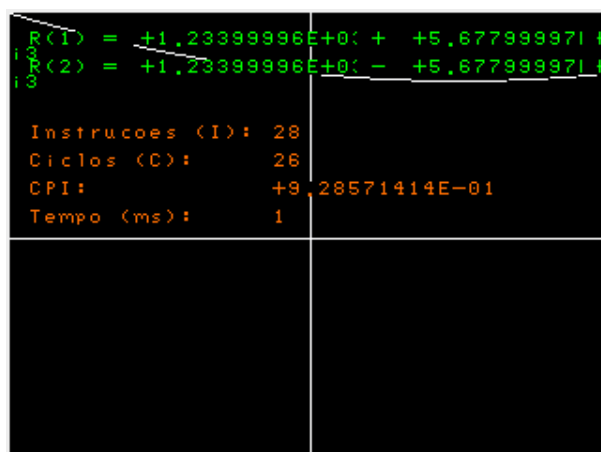


Figura 6: d) $[1, -2468, 33762440]$

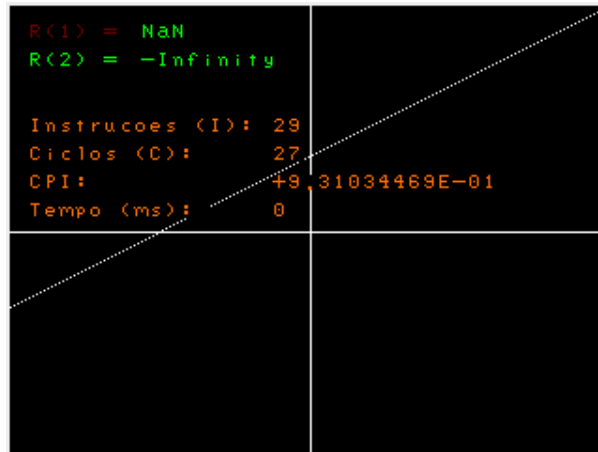


Figura 7: e) $[0, 10, 100]$

O desenvolvimento do algoritmo de auto-escalamento provou ser essencial para contornar as limitações físicas e de enquadramento do *Bitmap Display*. Além disso, a instrumentação do código principal com a leitura de registradores CSR (`instret`, `cycle` e `time`) permitiu validar, na prática, a teoria de desempenho de processadores. O experimento evidenciou um CPI próximo ao limite ideal para o pipeline clássico, além de demonstrar o tratamento determinístico e robusto do hardware perante exceções matemáticas severas, como a divisão por zero avaliada no Cenário E (resultando na sinalização padronizada de *NaN* e Infinito pela ULA de ponto flutuante).

Obs: Maior parte do código em si, foi basicamente para o desenho da parábola no BitMap. Por isso, para deixar mais visível e entendível, gastamos um pouco mais de linhas.

Conclusão Geral do Laboratório

O Laboratório 1 cumpriu de forma abrangente o seu propósito de ilustrar os fundamentos da Organização e Arquitetura de Computadores, unindo conceitos de software e hardware em um fluxo de trabalho contínuo. Através de etapas complementares, foi possível compreender a jornada completa de execução de um programa:

- Na primeira etapa, a análise assintótica e a derivação matemática das equações de tempo de execução (utilizando o algoritmo *Insertion Sort*) demonstraram como a organização dos dados de entrada afeta diretamente o esforço computacional no nível do ciclo de máquina.
- Na segunda etapa, a exploração da compilação cruzada (C para Assembly RISC-V) revelou o impacto dramático das otimizações de software. A análise das *flags* de compilação (`-O0`, `-O1`, `-O3`, `-Os`) comprovou empiricamente como um compilador inteligente pode reduzir o número total de instruções, substituir operações custosas (como multiplicações por deslocamentos de bit `slli`) e minimizar os acessos lentos à memória, otimizando o uso dos registradores temporários.
- Por fim, a programação direta em linguagem de montagem consolidou o entendimento prático sobre a manipulação do *datapath*. O controle direto sobre o hardware

evidenciou como a CPU interage com periféricos e processa dados em alta performance.

A experiência com o simulador RARS e a análise profunda das métricas de desempenho (I , C , CPI e Frequência) permitiram a constatação final de que o desempenho real de um sistema computacional não reside apenas no *clock* do processador, mas sim no equilíbrio entre a eficiência algorítmica, as estratégias de compilação e as características arquiteturais do hardware.

link para o repositório com os códigos implementados para a execução do laboratório
-> <https://github.com/guigas1/lab01-OAC-grupo-A1-.git>